# Learning Word Embeddings from Scratch

Skip-gram with Negative Sampling on the text8 Corpus

---

Natural Language Processing Project Report

---

**Instructor:** Nguyễn Kiêm Hiếu

**Group Members**

Nguyễn Bá Anh 20261218M
Nguyễn Thị Nhã Linh 20261262M

October 19, 2026

# Contents


**1 Introduction** **2**

**2 Background** **3**
- 2.1 Word representations . . . 3
- 2.2 Word2Vec: CBOW and Skip-gram . . . 3
- 2.3 Negative sampling . . . 4
- 2.4 Comparing embeddings . . . 4

**3 Methodology** **4**
- 3.1 Dataset . . . 4
- 3.2 Vocabulary and subsampling . . . 5
- 3.3 Training pairs . . . 5
- 3.4 Model, loss and gradients . . . 5
- 3.5 Negative sampling in practice . . . 5
- 3.6 Optimisation . . . 6

**4 Experimental Setup** **6**
- 4.1 Environment and runs . . . 6
- 4.2 Evaluation protocol . . . 7

**5 Results** **8**
- 5.1 Training loss (Q1) . . . 8
- 5.2 Nearest neighbours (Q2) . . . 8
- 5.3 Cosine similarity (Q3) . . . 9
- 5.4 Analogies (Q4) . . . 9
- 5.5 Embedding space (Q5) . . . 10
- 5.6 Effect of corpus size . . . 11

**6 Applications and Limitations** **11**
- 6.1 Applications . . . 11
- 6.2 Limitations . . . 12

**7 Conclusion** **12**

**A Concepts and their location in the code** **14**

**B Reproducing the results** **14**

**C Changes made to the original repository** **14**

# 1 Introduction

Most NLP models start by turning words into numbers. The simplest choice, a one-hot vector per word, treats every pair of words as equally different: *king* is as far from *queen* as it is from *carrot*. Word embeddings instead map each word to a dense vector of a few hundred dimensions, learned so that words used in similar contexts receive similar vectors (Harris, 1954; Firth, 1957; Bengio et al., 2003). Word2Vec (Mikolov et al., 2013a,b) made such embeddings cheap to train on large corpora and showed that simple vector arithmetic can expose relations between words (Mikolov et al., 2013c).

This project studies one Word2Vec variant, Skip-gram with Negative Sampling (SGNS), by training it from scratch and inspecting what it learns. It is organised around one question:

> *How can a model learn useful numerical representations of words from their surrounding context?*

We break this question into five experimental questions:

**Q1** Does training reduce the loss?

**Q2** Does the model learn semantically related words (nearest neighbours)?

**Q3** Are related words closer in vector space than unrelated ones (cosine similarity)?

**Q4** Do vector offsets represent relations (word analogies such as $king - man + woman$)?

**Q5** What does the embedding space look like in two dimensions?

**Source code and our contribution.** The SGNS implementation is taken from the open-source repository of Caselli (2026), written in NumPy without a deep learning framework. The original repository provides the model, the text8 loader, the vocabulary with subsampling, the training loop and basic similarity and analogy functions. On top of it we:

- made experiments reproducible: a random seed, all hyperparameters exposed on the command line, and a record of every run (hyperparameters, data statistics, runtime, environment and code version); we also fixed the documented training command, which did not run;
- profiled training and removed its main bottleneck in negative sampling, which made training about four times faster without changing the algorithm (Section 3.5);
- designed an evaluation protocol with queries fixed in advance, an untrained baseline and a frequency-matched baseline for cosine similarities (Section 4.2);
- ran a 1M-token and a 5M-token experiment and analysed the resulting embeddings.

Throughout the report, "the original repository" refers to the upstream code and "our experiment" to runs we configured and executed. All numbers in the tables are generated from the output files of our runs.

**Outline.** Section 2 introduces word embeddings, Skip-gram and negative sampling. Section 3 describes how the implementation realises them. Section 4 gives the experimental setup and evaluation protocol, and Section 5 the results. Section 6 discusses applications and limitations, and Section 7 concludes.

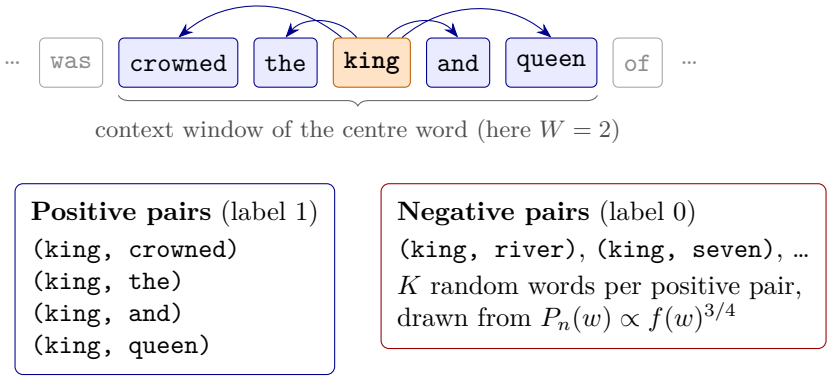



Figure 1: Training pairs for the centre word *king*. Each word inside the window forms a positive pair with the centre word; each positive pair is complemented by $K$ negative pairs with words drawn at random from the noise distribution (the negative words shown are illustrative).

# 2 Background

## 2.1 Word representations

Let $V$ be the size of the vocabulary. A one-hot representation encodes word $i$ as the $i$-th standard basis vector of $\mathbb{R}^V$. These vectors are sparse, as long as the vocabulary, and mutually orthogonal, so they carry no notion of similarity. A word embedding instead assigns each word $w$ a dense vector $\mathbf{v}_w \in \mathbb{R}^d$ with $d \ll V$ (here $d = 100$ and $V$ is in the tens of thousands).

Embeddings are learned from the *distributional hypothesis*: words that occur in similar contexts tend to have similar meanings (Harris, 1954), or in Firth's words, "you shall know a word by the company it keeps" (Firth, 1957). *King* and *queen* both appear near *crowned*, *reign* and *throne*; a model that has to predict such contexts is pushed to give the two words similar vectors. Neural language models learned embeddings as a by-product of next-word prediction (Bengio et al., 2003); Word2Vec keeps the idea but strips the model down so that it can be trained on far more text.

## 2.2 Word2Vec: CBOW and Skip-gram

Word2Vec (Mikolov et al., 2013a) is a shallow model with two embedding matrices: an input matrix $W_{\text{in}} \in \mathbb{R}^{V \times d}$ whose row $\mathbf{v}_w$ represents $w$ as a centre word, and an output matrix $W_{\text{out}} \in \mathbb{R}^{V \times d}$ whose row $\mathbf{v}'_w$ represents $w$ as a context word. The score of a (centre $c$, context $o$) pair is the dot product $\mathbf{v}'_o \cdot \mathbf{v}_c$. There are two architectures:

- **CBOW** (continuous bag-of-words) predicts the centre word from the average of its context vectors;
- **Skip-gram** predicts each context word from the centre word.

This project uses Skip-gram. Given a corpus $w_1, \ldots, w_T$ and a window size $c$, Skip-gram maximises the average log-probability of the context words:

$$\frac{1}{T} \sum_{t=1}^{T} \sum_{\substack{-c \le j \le c \\ j \neq 0}} \log p(w_{t+j} \mid w_t), \qquad p(o \mid c) = \frac{\exp(\mathbf{v}'_o \cdot \mathbf{v}_c)}{\sum_{w=1}^{V} \exp(\mathbf{v}'_w \cdot \mathbf{v}_c)}. \tag{1}$$

The softmax denominator sums over the whole vocabulary, so every training pair costs $O(V)$ operations. Figure 1 shows the pairs produced by one centre word.

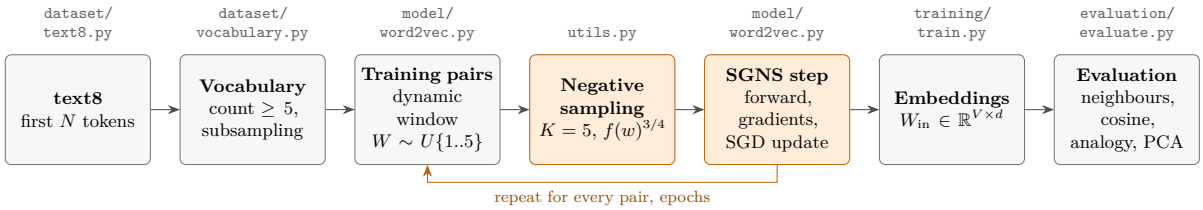



Figure 2: Training and evaluation pipeline. File names are relative to `src/`. The orange stages run once per training pair.

### 2.3 Negative sampling

Negative sampling (Mikolov et al., 2013b) replaces the $V$-way softmax with a binary task: tell the observed pair $(c, o)$ apart from $K$ pairs $(c, n_k)$ whose words $n_k$ are drawn from a noise distribution $P_n$. With $\sigma(x) = 1/(1 + e^{-x})$, the objective for one positive pair is

$$\log \sigma(\mathbf{v}'_o \cdot \mathbf{v}_c) + \sum_{k=1}^{K} \mathbb{E}_{n_k \sim P_n} \left[ \log \sigma(-\mathbf{v}'_{n_k} \cdot \mathbf{v}_c) \right], \qquad P_n(w) \propto f(w)^{3/4}, \tag{2}$$

where $f(w)$ is the frequency of $w$. The first term pulls the centre vector towards the vectors of its true context words; the second pushes it away from random words. Each pair now costs $O(K)$ instead of $O(V)$, with $K$ between 5 and 20 in practice. The exponent $3/4$ samples rare words more often than their raw frequency would. Goldberg and Levy (2014) give a detailed derivation of this objective.

The same paper introduces *subsampling* of frequent words: each occurrence of $w$ is discarded with probability $1 - \sqrt{t/f(w)}$ for a threshold $t$ (here $10^{-3}$). Very frequent words such as *the* carry little information about their neighbours, so dropping many of their occurrences speeds up training and gives rarer words more weight.

### 2.4 Comparing embeddings

Embeddings are compared with the cosine similarity

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \, \|\mathbf{v}\|} \in [-1, 1]. \tag{3}$$

The nearest neighbours of a word are the words with the highest cosine similarity to it. A word analogy "$a$ is to $b$ as $c$ is to ?" is answered by vector offsets (Mikolov et al., 2013c):

$$\underset{w \notin \{a,b,c\}}{\arg\max} \; \cos(\mathbf{v}_w, \; \mathbf{v}_b - \mathbf{v}_a + \mathbf{v}_c), \tag{4}$$

so *king* $-$ *man* $+$ *woman* $\approx$ *queen* corresponds to $a =$ *man*, $b =$ *king*, $c =$ *woman*.

## 3 Methodology

This section describes how the original repository implements SGNS and what we changed. Figure 2 gives the overall pipeline; Appendix A maps each concept to its location in the code.

### 3.1 Dataset

We use text8 (Mahoney, n.d.), a cleaned extract of English Wikipedia distributed for compression benchmarks: lower-case letters `a`–`z` separated by single spaces, about 17 million tokens in one line, with punctuation removed and digits spelled out as words. Tokenisation is therefore a

whitespace split. To keep training short, a run uses only the first $N$ tokens of the file (1,000,000 or 5,000,000 in our experiments), so it sees a contiguous prefix of the corpus rather than a random sample of articles. The corpus is downloaded automatically on first use.

### 3.2 Vocabulary and subsampling

The vocabulary contains every word that occurs at least 5 times in the $N$ tokens, indexed by decreasing frequency; other words are dropped from the token sequence. Frequent words are then subsampled as in Section 2.3: each occurrence of $w$ is kept with probability $\min\Big(1, \sqrt{t/f(w)}\Big)$, $t = 10^{-3}$, where $f(w)$ is the relative frequency of $w$ in the vocabulary. Dropping rare words and subsampling together leave about two thirds of the tokens (Table 2). It is applied once before training, so all epochs see the same subsampled sequence rather than a new draw per pass.

### 3.3 Training pairs

Pairs are generated on the fly. For each position $i$ of the subsampled sequence, a window size $W$ is drawn uniformly from $\{1, \dots, 5\}$ and the centre word is paired with the $W$ words on each side. Drawing $W$ at random weights nearby words more, since the immediate neighbours are inside the window for every $W$ while words five positions away are included only when $W = 5$. A centre word yields $W + 1$ context words on average, i.e. 6 with a maximum window of 5, so one epoch has about $6N'$ training pairs for $N'$ tokens after subsampling.

### 3.4 Model, loss and gradients

$W_{\text{in}}$ is initialised uniformly in $[-0.5/d, 0.5/d]$ and $W_{\text{out}}$ with zeros. For a centre word $c$, a positive context word $o$ and negative words $n_1, \dots, n_K$, let $s_o = \mathbf{v}'_o \cdot \mathbf{v}_c$ and $s_k = \mathbf{v}'_{n_k} \cdot \mathbf{v}_c$. The code minimises the negative of objective (2),

$$L = -\log \sigma(s_o) - \sum_{k=1}^{K} \log\big(1 - \sigma(s_k)\big), \tag{5}$$

which is the same quantity because $1 - \sigma(x) = \sigma(-x)$. Using $\sigma'(x) = \sigma(x)(1 - \sigma(x))$, the gradients are

$$\frac{\partial L}{\partial \mathbf{v}_c} = \big(\sigma(s_o) - 1\big)\,\mathbf{v}'_o + \sum_{k=1}^{K} \sigma(s_k)\,\mathbf{v}'_{n_k}, \tag{6}$$

$$\frac{\partial L}{\partial \mathbf{v}'_o} = \big(\sigma(s_o) - 1\big)\,\mathbf{v}_c, \qquad \frac{\partial L}{\partial \mathbf{v}'_{n_k}} = \sigma(s_k)\,\mathbf{v}_c. \tag{7}$$

Only $K+2$ rows of the two matrices change per step. Listing 1 shows the corresponding training step from the original repository, without its comments. `np.add.at` accumulates the updates correctly when the same negative word is drawn twice.

### 3.5 Negative sampling in practice

The noise distribution is the unigram distribution of the vocabulary raised to the power 3/4 and renormalised. Each positive pair receives $K = 5$ negatives; a negative that coincides with the centre or the context word is re-drawn.

The original repository drew each set of negatives with `np.random.choice(V, size=K, p=noise_dist)`. This call rebuilds the cumulative distribution over the whole vocabulary every time, so its cost grows with $V$ and is paid once per training pair. Profiling showed that it took most of the training time: throughput fell from about 10,900 steps per second with 13,966

Listing 1: One SGNS training step (`MyWord2Vec.step` in `src/model/word2vec.py`).

```
v_c = self.W_in[centre_idx]                    # (D)
v_o = self.W_out[pos_idx]                      # (D)
V_n = self.W_out[neg_idxs]                     # (K, D)

sig_o = sigmoid(v_c @ v_o)                     # P(positive | centre)
sig_k = sigmoid(V_n @ v_c)                     # (K)
loss = -np.log(sig_o + eps) - np.sum(np.log(1.0 - sig_k + eps))

grad_s_o = sig_o - 1.0                         # dL/ds_o
grad_s_k = sig_k                               # dL/ds_k
grad_vc = grad_s_o * v_o + grad_s_k @ V_n
grad_vo = grad_s_o * v_c
grad_Vn = np.outer(grad_s_k, v_c)              # (K, D)

self.W_in[centre_idx] -= lr * grad_vc          # SGD updates
self.W_out[pos_idx]   -= lr * grad_vo
np.add.at(self.W_out, neg_idxs, -lr * grad_Vn)
```

words to about 5,200 with 36,280 words, while the gradient step itself took only a small share. We replaced it with a sampler that draws one million i.i.d. negatives at a time from the same distribution and hands them out in order. The samples follow the same distribution: over two million draws, their total variation distance to $P_n$ was 0.041, identical to that of direct sampling, which is pure sampling noise. Throughput became 41,000–46,000 steps per second in our runs, almost independent of vocabulary size (Table 2), and the 1M-token, three-epoch configuration took 251 seconds instead of 1,023 seconds with the original code. Both versions reached the same per-epoch losses (Section 5.1), confirming that training behaves the same.

### 3.6 Optimisation

Training uses plain stochastic gradient descent on one pair at a time. The learning rate decays linearly from $\eta_0 = 0.025$ towards $10^{-4}$ over the estimated number of steps epochs $\times N' \times 6$:

$$\eta_t = \max\bigl(10^{-4},\ \eta_0\,(1 - t/T)\bigr)\,. \tag{8}$$

After training, the rows of $W_{\text{in}}$ serve as the word embeddings; $W_{\text{out}}$ is used only during training.

## 4 Experimental Setup

### 4.1 Environment and runs

All runs were executed on one core of an Apple M5 laptop with Python 3.11.17 and NumPy 2.4.6, from commit `23fa9c2` of our fork. Each run writes its embeddings, its loss history and a `config.json` file with every hyperparameter, the vocabulary size, the number of tokens after subsampling, the training time and the environment; the tables in this report are generated from these files. Two runs with the same arguments produce identical embeddings.

We report two runs that differ only in the amount of text:

- **Demo**: the first 1,000,000 tokens of text8, used for quick iterations and as a smaller-corpus comparison;
- **Final**: the first 5,000,000 tokens (about 29% of text8), our main result.

| Hyperparameter | Value | CLI option |
|---|---|---|
| Tokens of text8 used | 1,000,000 (demo) / 5,000,000 (final) | `--max-tokens` |
| Minimum word count | 5 | `--min-count` |
| Subsampling threshold $t$ | 0.001 | `--subsample-t` |
| Embedding dimension $d$ | 100 | `--embed-dim` |
| Epochs | 3 | `--epochs` |
| Maximum window size | 5 | `--max-window` |
| Negative samples $K$ | 5 | `--n-negatives` |
| Noise distribution exponent | 0.75 | fixed |
| Initial / minimum learning rate | 0.025 / 0.0001 | `--start-lr`, `--lr-min` |
| Loss report interval (steps) | 100,000 / 500,000 | `--report-every` |
| Random seed | 42 | `--seed` |

Table 1: Hyperparameters shared by both runs (only the number of tokens and the loss report interval differ).

Table 1 lists the hyperparameters. The window size, number of negatives, subsampling threshold and learning rate follow common Word2Vec defaults; we did not tune them. A smoke test on 100,000 tokens was used only to check the pipeline and is not reported.

### 4.2 Evaluation protocol

Training loss shows whether the objective is being optimised, but not whether the embeddings are meaningful, so we evaluate the embeddings directly. All evaluation uses cosine similarity on the rows of $W_{\text{in}}$.

- **Training loss (Q1).** The loss of each pair is computed before its update. We record its mean over consecutive windows of steps (100,000 for the demo run, 500,000 for the final run) and the mean over each epoch. The original repository recorded a running average since the start of the epoch, which lags behind the current loss and resets at every epoch.
- **Nearest neighbours (Q2).** The eight words closest to each of 8 query words.
- **Cosine similarity (Q3).** The similarity of 6 related pairs and 3 unrelated control pairs. Raw cosine values are hard to interpret, because word vectors share a common direction (Mu and Viswanath, 2018): in our final run, random pairs of frequent words already average 0.32. We therefore report each value as a percentile: the share of 10,000 random pairs of words that are less similar. The random pairs are drawn from the 2,000 most frequent words, which contain every query word.
- **Analogies (Q4).** 8 analogies of the form $a : b :: c :?$, answered with Equation (4). We report the rank of the expected answer among the ten best candidates, and hit@1 and hit@5 counts.
- **Visualisation (Q5).** A two-dimensional principal component analysis (PCA) of 32 words from four groups (countries, numbers, months, family and royalty). The vectors are L2-normalised and centred before the decomposition.
- **Untrained baseline.** Every measurement is repeated on the same model before its first update, rebuilt from the run's seed. Anything the untrained model also shows cannot be credited to training.

| | Demo | Final |
|---|---|---|
| Tokens of text8 | 1,000,000 | 5,000,000 |
| Vocabulary size | 13,966 | 36,280 |
| Tokens after subsampling | 635,140 | 3,372,979 |
| Training steps (pairs) | 11,425,544 | 60,708,682 |
| Training time | 4.2 min | 24.7 min |
| Throughput (steps/s) | 45,500 | 40,900 |
| Mean loss, epoch 1 | 2.591 | 2.347 |
| Mean loss, epoch 2 | 2.424 | 2.250 |
| Mean loss, epoch 3 | 2.468 | 2.299 |
| Random-pair cosine (top 2,000) | 0.61 ± 0.19 | 0.32 ± 0.12 |
| Analogies: hit@1 / hit@5 | 0 / 2 of 8 | 4 / 6 of 8 |

Table 2: Summary of the two runs. The only difference in configuration is the number of tokens.

**Fixed queries and one deviation.** All query words, pairs, analogies and PCA groups were written into the evaluation code before any trained model was inspected, and every result is reported, including failures. One part of the protocol changed after we saw results. The random-pair baseline was first drawn from the whole vocabulary. On the demo run it averaged 0.84, against 0.61 among the 2,000 most frequent words, because rare words, which make up most of the vocabulary, receive few updates and stay close to a shared direction (pairs of words seen at most 19 times averaged 0.93). Against that inflated baseline, related pairs looked unremarkable. Since all query words are frequent, we switched to the frequency-matched baseline above; the queries themselves did not change.

# 5 Results

Unless stated otherwise, results refer to the final run (5M tokens). Table 2 summarises both runs.

## 5.1 Training loss (Q1)

Figure 3 shows that the loss falls quickly at the start of training, from 3.57 in the first window of the final run (4.07 in the demo run) to about 2.3 (about 2.45 in the demo run), and is nearly flat afterwards. Most of the decrease happens in the first quarter of the first epoch. The two curves are not directly comparable, because the final run sees text that the demo run never sees.

The epoch means in Table 2 are more surprising. Every epoch passes over the same text in the same order, so they are comparable within a run: the loss drops from epoch 1 to epoch 2 but *rises* again in epoch 3, in both runs (2.250 to 2.299 for the final run). This is not caused by our modifications: the original code on the demo configuration gives the same pattern (2.425 to 2.469). We have not established the cause. One possible explanation is that the loss is measured on each pair just before learning from it: with a large learning rate, the model adapts to the article it is reading and predicts the next pairs of that article better, an advantage that fades as the learning rate decays towards zero. Whatever the reason, the answer to Q1 is that the objective is clearly optimised early on, but the online training loss says little about how good the embeddings are, which is why we evaluate them directly.

## 5.2 Nearest neighbours (Q2)

Table 3 shows that training produces meaningful neighbourhoods, while the untrained model returns unrelated words. The clearest cases are closed classes of words that can replace each

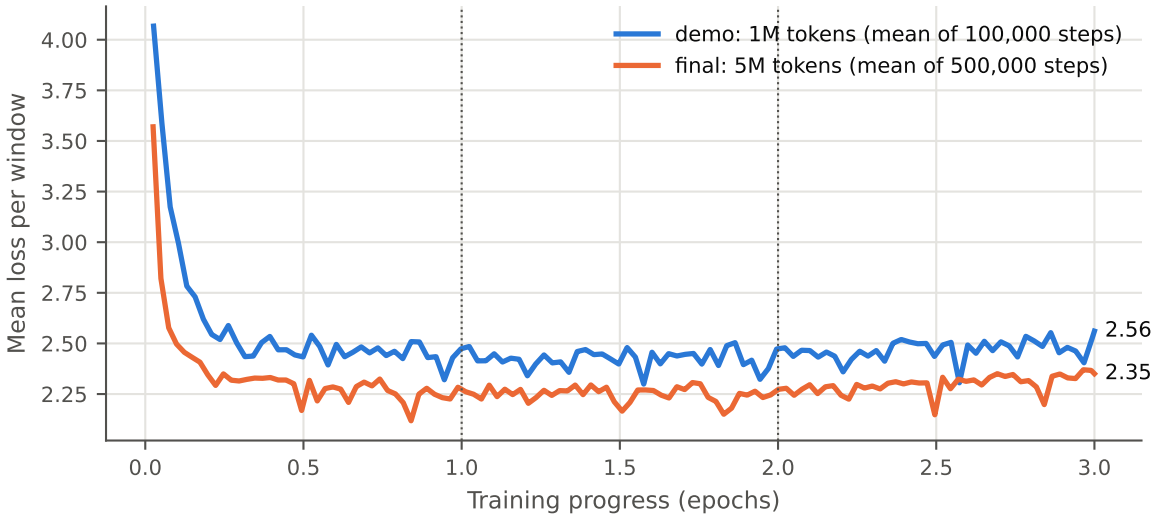



Figure 3: Mean training loss per window of steps. Dotted lines mark epoch boundaries; the epochs of a run have approximately equal numbers of steps.

other in a sentence: all eight neighbours of *january* are months, those of *france* are European countries, and the first five neighbours of *three* are numbers. Other lists mix words of the same type with topically related words: *music* has genres (*jazz*, *pop*, *folk*), *computer* has *hardware*, *software* and *programmer*, and *war* has names of wars together with *outbreak* and *veterans*.

Some neighbours are less intuitive. *King* is closest to specific monarchs and regnal terms (*alexius*, *throne*, *vii*, *honorius*) rather than to *queen*, and the neighbours of *three* end with rare tokens (*hdi*, *unpaved*) that probably come from tables of figures in country articles. With fewer tokens, the demo run had more such noise: its neighbours of *three* were mostly abbreviations, and those of *computer* included *animation* and *game*.

### 5.3 Cosine similarity (Q3)

In the final run, every related pair is more similar than at least 85% of random pairs, and five of the six are more similar than over 99% of them (Table 4). The control pairs have percentiles between 5 and 32. Before training, the percentiles are scattered with no relation to meaning. The answer to Q3 is therefore yes, provided similarities are compared with the right reference.

The raw values show why that reference matters. The cosine of *king* and *queen* *decreases* from the demo to the final run (0.91 to 0.70) while its percentile increases (from 98.2 to 99.7), because the typical similarity of frequent words fell from 0.61 to 0.32: with more data, vectors spread out and share less of a common direction. Raw cosine values are thus not comparable across models, and a value such as 0.70 means little on its own.

### 5.4 Analogies (Q4)

The final model ranks the expected answer first for 4 of the 8 analogies and within the top five for 6 (Table 5). The demo model reaches 0 and 2, and the untrained model none. Capital cities (*paris* $\rightarrow$ *berlin*), family relations (*mother*), comparatives (*smaller*) and nationality adjectives (*french*) work. The best-known example, *king* $-$ *man* $+$ *woman*, does not: *queen* is not among the ten best answers, which start with *daughter* and two historical rulers, even though *king* and *queen* are highly similar. The analogy *france* : *paris* :: *italy* : *rome* also fails, while the same relation works for Germany.

An offset such as *king* $-$ *man* only transfers to *woman* if the relation is expressed consistently

| Query | Trained (top 8) | Untrained (top 3) |
|---|---|---|
| *king* | alexius, grandson, throne, vii, emperor, pileser, aram, honorius | marpat, verbally, brando |
| *computer* | computers, computing, interface, hardware, programmer, laptop, simulation, software | subunits, svenska, mausoleum |
| *city* | town, edmonton, bronx, bremerhaven, suburb, kansas, aalborg, metropolitan | expect, affinity, sharp |
| *france* | italy, spain, netherlands, switzerland, germany, belgium, portugal, sweden | implying, theravada, dummett |
| *january* | february, december, october, june, november, july, march, april | cajun, troopers, electrophilic |
| *three* | seven, four, six, two, five, hdi, unpaved, sfg | gropius, washes, claudius |
| *war* | peloponnesian, ww, wwii, outbreak, veterans, boer, crimean, falklands | ashurbanipal, route, monophyletic |
| *music* | jazz, musical, dance, pop, funk, folk, hop, genres | accelerators, revolutionaries, ayckbourn |

Table 3: Nearest neighbours by cosine similarity, final run and the same model before training. All 8 query words were fixed in advance.

| Pair | Demo (1M) | Final (5M) | Untrained |
|---|---|---|---|
| *king* – *queen* | 0.91 (p98) | 0.70 (p100) | 0.10 (p82) |
| *man* – *woman* | 0.89 (p97) | 0.77 (p100) | −0.06 (p30) |
| *computer* – *software* | 0.80 (p84) | 0.75 (p100) | 0.17 (p96) |
| *city* – *country* | 0.78 (p81) | 0.45 (p85) | −0.14 (p9) |
| *france* – *germany* | 0.86 (p93) | 0.76 (p100) | 0.05 (p68) |
| *january* – *february* | 0.95 (p100) | 0.91 (p100) | −0.03 (p39) |
| *king* – *software* | 0.14 (p1) | 0.12 (p5) | 0.00 (p49) |
| *computer* – *january* | 0.42 (p17) | 0.18 (p12) | 0.04 (p67) |
| *music* – *france* | 0.42 (p18) | 0.25 (p32) | 0.05 (p69) |

Table 4: Cosine similarity of fixed word pairs, with its percentile pN among random pairs of the 2,000 most frequent words of the same model. Above the line: related pairs; below: unrelated control pairs. "Untrained" is the final model before training.

across many contexts. Possible reasons for the failures are the small and topically narrow corpus, in which *king* occurs mostly in historical articles about specific rulers (as its neighbours suggest), and the small number of examples of each relation. Mikolov et al. (2013a) trained on corpora orders of magnitude larger. With 8 analogies, these counts illustrate the trend from 1M to 5M tokens but are not an accuracy estimate.

### 5.5 Embedding space (Q5)

Figure 4 projects the 32 words of the four groups onto their first two principal components. After training, the groups form four separate clusters: numbers and months are very tight, countries are spread more widely, and family and royalty words lie together, with *king*, *queen* and *prince* next to each other. The two components capture 27% and 16% of the variance of these vectors. Before training, the same words show no grouping and the components capture only 8% and 7%, as expected for random vectors. The picture should be read with care: the 32 words were chosen to belong to distinct groups, and two components show only part of the 100-dimensional structure.

| Analogy | Expected | Final: top 3 | Rank of expected | | |
|---|---|---|---|---|---|
| | | | Final | Demo | Untrained |
| *man* : *king* :: *woman* : ? | *queen* | daughter, caligula, amalric | >10 | >10 | >10 |
| *france* : *paris* :: *italy* : ? | *rome* | interred, venice, geneva | >10 | >10 | >10 |
| *france* : *paris* :: *germany* : ? | *berlin* | berlin, munich, exposition | 1 | >10 | >10 |
| *man* : *woman* :: *boy* : ? | *girl* | baby, girl, marple | 2 | >10 | >10 |
| *brother* : *sister* :: *father* : ? | *mother* | mother, grandfather, parents | 1 | >10 | >10 |
| *good* : *better* :: *bad* : ? | *worse* | fare, worse, cheaper | 2 | >10 | >10 |
| *big* : *bigger* :: *small* : ? | *smaller* | smaller, parking, forested | 1 | 3 | >10 |
| *england* : *english* :: *france* : ? | *french* | french, spanish, portuguese | 1 | 5 | >10 |

Table 5: Analogies $a : b :: c :?$ answered by the nearest word to $\mathbf{v}_b - \mathbf{v}_a + \mathbf{v}_c$. Rank of the expected answer among the ten best candidates (>10: not among them).

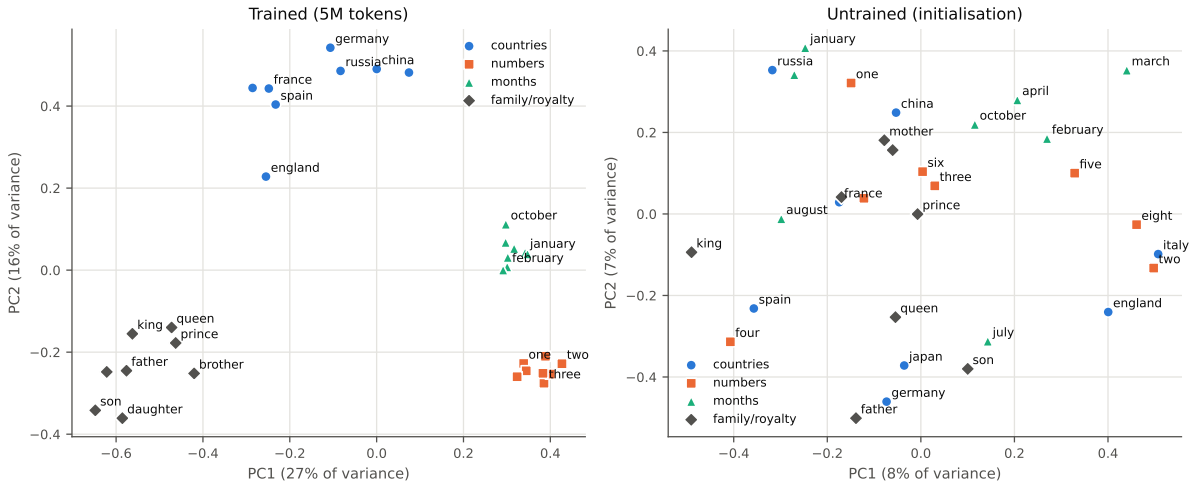



Figure 4: PCA of 32 words from four groups fixed in advance, final model (left) and the same model before training (right). Some labels in dense clusters are omitted to avoid overlap.

### 5.6 Effect of corpus size

The demo and final runs share every setting except the number of tokens. Five times more text gave a vocabulary 2.6 times larger, about 5.3 times more training pairs and 5.9 times the training time (Table 2). Every evaluation improved: neighbour lists contain less noise, related and unrelated pairs are better separated, and the analogy hits went from 0/2 to 4/6 (hit@1/hit@5). Because more text also means more updates, these two factors cannot be separated with our runs.

## 6 Applications and Limitations

### 6.1 Applications

Embeddings like these are used wherever a model needs a notion of word similarity without labelled data:

- **Input features for downstream models.** Pre-trained vectors can initialise the embedding layer of a text classifier, for example for sentiment analysis, so that words unseen in the labelled training set still receive meaningful inputs.

- **Similarity search and query expansion.** Nearest neighbours suggest related terms; for instance, a search for *january* can be expanded with other months.
- **Exploring a corpus.** Neighbour lists and two-dimensional maps such as Figure 4 summarise which words a corpus uses in similar ways.
- **Beyond words.** The same objective applies to any sequence of discrete items, such as products in a shopping session, by treating co-occurring items as context.

Our embeddings, trained on 5,000,000 tokens, are a teaching artefact rather than a resource for these uses; practical systems train on corpora several orders of magnitude larger.

### 6.2 Limitations

- **Corpus size and coverage.** The final run uses about 29% of text8, and as a contiguous prefix it covers only the first articles of the dump. Vocabulary, neighbours and analogy results depend on which topics appear in that prefix.
- **Rare words.** Words seen only a few times receive few updates and stay close to a direction shared by all such words. Their neighbours are unreliable, and they inflate any similarity statistic computed over the whole vocabulary (Section 4.2).
- **One vector per word.** A static embedding merges all senses of a word (*bank* as a river side and as an institution) and ignores word order beyond the window.
- **Small, qualitative evaluation.** We use 8 neighbour queries, 9 word pairs and 8 analogies. They show clear tendencies but not reliable rates; a standard benchmark such as a word-similarity dataset or the full Google analogy set would give quantitative scores.
- **Training loss is measured on the training stream.** Each pair's loss is computed just before the model learns from it; there is no held-out objective, and the curve depends on the order of the text.
- **Single seed and no tuning.** Each configuration was trained once with common default hyperparameters, so we cannot quantify run-to-run variation or claim that the settings are good for this corpus. Subsampling is drawn once rather than per epoch.

## 7 Conclusion

Word2Vec learns a vector for each word from the words around it. In this project, a NumPy implementation of Skip-gram with Negative Sampling, adapted from an open-source repository, was trained on the first 5,000,000 tokens of text8, and the resulting embeddings were evaluated with nearest neighbours, cosine similarities, analogies and a two-dimensional projection, each compared with the same model before training. The answers to the five questions are:

**Q1** **Loss.** The loss drops sharply early in training and then stays nearly flat; the per-epoch means even rise in the last epoch. The online training loss confirms that the objective is optimised but is a poor guide to embedding quality.

**Q2** **Neighbours.** Yes. Words of the same type cluster (months, numbers, countries, music genres), together with topically related words; the untrained model gives unrelated words.

**Q3** **Similarity.** Yes, against the right reference. Among random pairs of frequent words, related pairs reach a percentile of at least 85 and unrelated pairs at most 32. Raw cosine values alone are misleading and not comparable between models.

**Q4 Analogies.** Partly. 4 of 8 fixed analogies are solved exactly and 6 within the top five, up from 0 and 2 with 1M tokens; $king - man + woman \approx queen$ is not.

**Q5 Embedding space.** Four groups of words chosen in advance form separate clusters in a two-dimensional PCA, which is not the case before training.

Beyond these answers, the project showed two practical points. Profiling a from-scratch implementation can uncover a large, easy speed-up: batching the negative samples made training about four times faster without changing the algorithm. And evaluating embeddings requires baselines: comparing with the untrained model and with frequency-matched random pairs turned plausible-looking numbers into evidence.

Natural next steps are to train on the full text8 corpus, to measure performance on standard word-similarity and analogy benchmarks, to repeat each configuration with several seeds, and to compare the input vectors with the sum of input and output vectors.

## References


Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin. A neural probabilistic language model. *Journal of Machine Learning Research*, 3:1137–1155, 2003.

Niccolò Caselli. word2vec-from-scratch: Skip-gram Word2Vec with negative sampling in NumPy. https://github.com/NiccoloCase/word2vec-from-scratch, 2026. GitHub repository, commit ff07bfb.

John R. Firth. A synopsis of linguistic theory, 1930–1955. In *Studies in Linguistic Analysis*, pages 1–32. Blackwell, Oxford, 1957.

Yoav Goldberg and Omer Levy. word2vec explained: Deriving Mikolov et al.'s negative-sampling word-embedding method. arXiv:1402.3722, 2014.

Zellig S. Harris. Distributional structure. *Word*, 10(2–3):146–162, 1954.

Matt Mahoney. About the test data (text8). http://mattmahoney.net/dc/textdata.html, n.d. Accessed October 2026.

Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. In *Workshop Track of the 1st International Conference on Learning Representations (ICLR)*, 2013a. arXiv:1301.3781.

Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S. Corrado, and Jeff Dean. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems 26 (NeurIPS)*, pages 3111–3119, 2013b. arXiv:1310.4546.

Tomas Mikolov, Wen-tau Yih, and Geoffrey Zweig. Linguistic regularities in continuous space word representations. In *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 746–751, 2013c.

Jiaqi Mu and Pramod Viswanath. All-but-the-top: Simple and effective postprocessing for word representations. In *International Conference on Learning Representations (ICLR)*, 2018.

# A Concepts and their location in the code

| Concept | Code (relative to `src/`) |
|---|---|
| text8 download and loading | `dataset/text8.py` |
| Vocabulary (minimum count, frequency order) | `dataset/vocabulary.py`, `Vocabulary._build` |
| Subsampling of frequent words | `dataset/vocabulary.py`, `encode_subsampled` |
| Data pipeline (load, vocabulary, subsample) | `dataset/training.py`, `get_training_data` |
| Training pairs (dynamic window), epochs | `model/word2vec.py`, `MyWord2Vec.train` |
| Noise distribution, batched negative sampler | `utils.py`, `build_noise_distribution`, `NegativeSampler` |
| Forward pass, gradients, SGD update | `model/word2vec.py`, `MyWord2Vec.step` |
| Learning-rate decay | `utils.py`, `learning_rate_decay` |
| Run orchestration, seed, saving, `config.json` | `training/train.py` |
| Nearest neighbours, analogies | `model/word2vec.py`, `get_most_similar`, `analogy` |
| Evaluation protocol, PCA figure | `evaluation/evaluate.py` |

Table 6: Where each part of the method is implemented.

# B Reproducing the results

```
conda env create -f environment.yml
conda activate word2vec-from-scratch

experiments/run_demo.sh      # train + evaluate, 1M tokens x 3 epochs
experiments/run_final.sh     # train + evaluate, 5M tokens x 3 epochs
python experiments/make_report_assets.py   # figures and tables of this report

python demo/demo.py          # interactive queries on the final embeddings
```

Each run writes `output/<run>/config.json`, `evaluation.json` and `evaluation.md`; the report is compiled with XeLaTeX and BibTeX from `report/main.tex`.

# C Changes made to the original repository

- **Command line.** The documented command `python run.py train …` failed because the CLI library treated the single command as the root command; `train` and `evaluate` are now subcommands. Previously hard-coded hyperparameters (minimum count, subsampling threshold, learning rates, report interval) are options, and `--seed` seeds both random number generators.
- **Run record.** Each run writes `config.json` (hyperparameters, data statistics, loss history, runtime, Python and NumPy versions, git commit).
- **Loss logging.** Mean loss per window of steps and per epoch, instead of a running average since the start of the epoch.
- **Negative sampling.** Batched sampling from the same distribution (Section 3.5).
- **Analogy scores.** The target vector $\mathbf{v}_b - \mathbf{v}_a + \mathbf{v}_c$ is normalised, so scores are cosine similarities; the original scores could exceed 1. Rankings are unchanged.

- **Evaluation.** New `evaluate` command with fixed queries, an untrained baseline, frequency-matched cosine percentiles and a PCA figure; experiment scripts and a demo script.

The model, its initialisation, the loss, the gradients, the window sampling and the learning-rate schedule are those of the original repository.